

Git and GitHub for Beginners

A terminal-first handout for version control, collaboration, and reproducible work

Teaching material

2026-07-07

Table of contents

1	Learning Goals	3
2	Part 0: Command-Line Basics	3
2.1	Reading a Prompt	4
2.2	Files, Folders, and Paths	4
2.3	Common CLI Commands	5
2.4	Writing Small Files from the Terminal	6
2.5	Safe Terminal Habits	7
3	Before Class Setup	7
4	Why Git Exists	8
5	A Mental Model	9
6	Part 1: Create a Local Repository	9
7	Part 2: Check Repository Status	10
8	Part 3: Create and Track a File	11
9	Part 4: Edit, Diff, Stage, Commit	12
10	Part 5: What Is a Commit?	13
11	Part 6: The Staging Area	13
12	Part 7: Branches	14

13 Part 8: GitHub Remote Workflow	15
14 Part 9: Clone an Existing Repository	16
15 Part 10: Pull and Push	17
16 Part 11: Pull Requests	17
17 Part 12: Merge Conflicts	18
18 Part 13: .gitignore	19
19 Part 14: Safe Undo Commands	20
20 Part 15: Reading History	21
21 Part 16: Working With VS Code	21
22 Part 17: GitHub Issues and Project Work	22
23 Part 18: Good Repository Hygiene	23
24 Part 19: Common Beginner Problems	23
24.1 “Git says not a git repository”	23
24.2 “The command line says no such file or directory”	24
24.3 “code command does not work”	24
24.4 “Nothing to commit”	25
24.5 “Rejected push”	25
24.6 “Authentication failed”	25
25 Part 20: Responsible AI Use With Git	26
26 Class Lab	26
26.1 Lab A: Local Repository	26
26.2 Lab B: Branch and Merge	27
26.3 Lab C: GitHub	27
26.4 Lab D: Pull Request	28
27 Quick Reference	28
27.1 Terminal Commands	28
27.2 Git Commands	29
28 Exit Ticket	30
29 Suggested Homework	30

Instructor note

This handout is written as a Quarto source file (`.qmd`), but it is no longer designed as a runnable notebook. The command blocks are teaching examples for a live terminal demonstration.

Use **Git Bash** on Windows, **Terminal** on macOS, or the integrated terminal in **Visual Studio Code**. Students should run commands one at a time and compare their terminal output with the explanation in class.

1 Learning Goals

By the end of this lesson, students should be able to:

1. Explain what a command-line interface is and why Git is usually taught from a terminal.
2. Navigate folders and inspect files with common terminal commands.
3. Explain what Git tracks and why version control is useful.
4. Use the basic local Git workflow: `status`, `add`, `commit`, `log`, and `diff`.
5. Create and switch branches.
6. Merge changes and understand why conflicts happen.
7. Connect a local repository to GitHub.
8. Clone, pull, push, and open a pull request.
9. Use `.gitignore` and safe undo commands.
10. Describe a practical daily workflow for individual and group projects.

2 Part 0: Command-Line Basics

Git is often taught in a command-line interface, or **CLI**, because the commands are precise, repeatable, and easy to document.

A terminal has two parts:

Term	Meaning
Terminal app	The window where you type commands.
Shell	The program that interprets your commands.
Prompt	The text shown before the place where you type.
Current working directory	The folder where your commands are currently running.

On Windows, this lesson uses **Git Bash**, which is installed with Git for Windows. Git Bash gives Windows students a Unix-like command style that is close to macOS and Linux.

On macOS, this lesson uses **Terminal**. The default macOS shell is usually **zsh**, but the basic commands in this handout work the same way for beginner Git use.

VS Code option

In Visual Studio Code, use **Terminal > New Terminal**. On Windows, choose **Git Bash** as the terminal profile if it is available. On macOS, the default terminal profile is fine.

2.1 Reading a Prompt

A Git Bash prompt might look like this:

```
student@LAPTOP MINGW64 ~/Documents
$
```

A macOS Terminal prompt might look like this:

```
student@MacBook Documents %
```

The prompt shows context. You type the command after the prompt. In this handout, command blocks do **not** include \$ or % so students can copy only the command itself.

2.2 Files, Folders, and Paths

The command line always has a current folder. Use **pwd** to print it:

```
pwd
```

Common path symbols:

Symbol	Meaning
.	the current folder
..	the parent folder
~	your home folder
/	folder separator in Git Bash and macOS Terminal

Windows File Explorer often shows paths with backslashes, such as:

```
C:\Users\Student\Documents
```

Git Bash uses Unix-style paths instead:

```
/c/Users/Student/Documents
```

Most beginner commands are the same in Git Bash and macOS Terminal:

```
cd ~/Documents
pwd
ls
```

If a folder name contains spaces, wrap the path in quotes:

```
cd "My Class Project"
```

2.3 Common CLI Commands

Task	Git Bash on Windows	macOS Terminal	Notes
Show current folder	<code>pwd</code>	<code>pwd</code>	Short for “print working directory.”
List files	<code>ls</code>	<code>ls</code>	Shows visible files and folders.
List all files	<code>ls -la</code>	<code>ls -la</code>	Includes hidden files such as <code>.git</code> .
Change folder	<code>cd folder-name</code>	<code>cd folder-name</code>	Moves into a folder.
Go to home folder	<code>cd ~</code>	<code>cd ~</code>	Returns to your user folder.
Go up one folder	<code>cd ..</code>	<code>cd ..</code>	Moves to the parent folder.
Make a folder	<code>mkdir</code>	<code>mkdir</code>	Creates a new folder.
Create an empty file	<code>git-github-lab</code> <code>touch README.md</code>	<code>git-github-lab</code> <code>touch README.md</code>	Useful for quick practice files.
Open file in VS Code	<code>code README.md</code>	<code>code README.md</code>	Requires the VS Code command-line tool.

Task	Git Bash on Windows	macOS Terminal	Notes
Open current folder in VS Code	<code>code .</code>	<code>code .</code>	The dot means current folder.
View a text file	<code>cat README.md</code>	<code>cat README.md</code>	Prints the file in the terminal.
Copy a file	<code>cp old.txt new.txt</code>	<code>cp old.txt new.txt</code>	Copies from source to destination.
Rename or move a file	<code>mv old.txt new.txt</code>	<code>mv old.txt new.txt</code>	Same command for rename and move.
Delete a file	<code>rm scratch.txt</code>	<code>rm scratch.txt</code>	Use carefully; this does not use the recycle bin.
Remove an empty folder	<code>rmdir old-folder</code>	<code>rmdir old-folder</code>	Works only when the folder is empty.
Clear the terminal	<code>clear</code>	<code>clear</code>	Removes previous output from view.
Open current folder in file manager	<code>explorer .</code>	<code>open .</code>	Windows uses Explorer; macOS uses Finder.

Be careful with deletion commands

For beginners, avoid `rm -r`, `rm -rf`, and `git clean -fd` unless an instructor explains exactly what will be deleted. These commands can remove folders and untracked files permanently.

2.4 Writing Small Files from the Terminal

For teaching, it is usually clearer to create or edit files in VS Code:

```
code README.md
```

Sometimes instructors may show quick terminal examples:

```
echo "# Git Practice" > README.md
echo "This repository is for learning Git." >> README.md
cat README.md
```

The difference matters:

Command	Meaning
>	write output to a file, replacing the file if it already exists
>>	append output to the end of a file

2.5 Safe Terminal Habits

Before running Git commands, students should learn this rhythm:

```
pwd
ls
git status
```

This answers three questions:

1. Where am I?
2. What files are here?
3. Is this a Git repository, and what changed?

Run one command at a time during practice. If the output is confusing, stop and read it before trying another command.

3 Before Class Setup

Students should install:

- Git: <https://git-scm.com/>
- Visual Studio Code: <https://code.visualstudio.com/>
- A GitHub account: <https://github.com/>
- Optional: GitHub Desktop: <https://desktop.github.com/>

After installing Git, check the version:

```
git --version
```

Configure your identity once on your own computer:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

The name and email are stored in commits. Use the same email address that you use for GitHub, or use the private GitHub no-reply email from your GitHub account settings.

💡 Windows note

On Windows, Git usually installs Git Bash and Git Credential Manager. Git Credential Manager helps with browser-based GitHub sign-in when pushing over HTTPS.

4 Why Git Exists

Many students start with file names like these:

```
report.docx
report_final.docx
report_final_revised.docx
report_final_revised_REAL.docx
report_final_revised_REAL_2.docx
```

This is version control by file name. It works for a short time, but it becomes confusing when:

- several people edit the same project,
- you need to know who changed what,
- you want to undo one specific change,
- you need to compare two versions,
- your project contains code, data, figures, notebooks, and written reports.

Git solves this by recording the history of a project in a structured way.

Git is a local version control system. It runs on your computer and tracks changes in a folder called a repository.

GitHub is an online hosting and collaboration platform for Git repositories. It adds remote backup, issue tracking, pull requests, code review, project management, and sharing.

! Git and GitHub are not the same thing

Git is the version control tool.

GitHub is a website and cloud service that stores Git repositories and helps people collaborate.

5 A Mental Model

Git has four common places where your work can be:

Place	Meaning	Typical command
Working tree	Files you are currently editing	edit files
Staging area	Changes selected for the next commit	<code>git add</code>
Local repository	Saved commits on your computer	<code>git commit</code>
Remote repository	Copy on GitHub or another server	<code>git push</code> , <code>git pull</code>

The core workflow is:

```
edit files -> stage changes -> commit locally -> push to GitHub
```

In commands:

```
git status
git add README.md
git commit -m "Describe the change"
git push
```

i Git is not automatic save

Editing a file is not enough. Git records a version only when you stage and commit the change.

6 Part 1: Create a Local Repository

A Git repository is a folder with a hidden `.git` directory inside it. The `.git` directory stores the project's history.

Start in a safe place such as `Documents`, then create a practice folder:

```
cd ~/Documents
mkdir git-github-lab
cd git-github-lab
git init
git branch -M main
git status
```

After `git init`, the folder becomes a Git repository.

Use `ls -la` to show hidden files:

```
ls -la
```

You should see a hidden `.git` folder.

For a classroom practice repository, an instructor may configure a local teaching identity. This affects only this repository:

```
git config --local user.name "Student Example"
git config --local user.email "student@example.com"
git config --local --list
```

Repository scope

`git config --global` changes your user settings for all repositories on your computer.
`git config --local` changes settings only for the current repository.

7 Part 2: Check Repository Status

`git status` is the safest Git command. Use it often.

```
git status
```

It answers:

- Which branch am I on?
- Are there untracked files?
- Are there modified files?
- Are there staged changes waiting to be committed?
- Is my local branch ahead of or behind the remote branch?

💡 Beginner habit

When in doubt, run:

```
git status
```

8 Part 3: Create and Track a File

Create a README.md file:

```
code README.md
```

Type this content, save the file, and return to the terminal:

```
# Git and GitHub Practice

This repository is for learning Git.

Today I will practice:

- checking status
- staging changes
- committing changes
- reading history
```

Check status:

```
git status
```

Git notices the file, but it does not automatically track it. The file is currently untracked.

Stage the file:

```
git add README.md
git status
```

Commit the file:

```
git commit -m "Add README"
git log --oneline
```

💡 Commit message habit

A useful commit message finishes this sentence:

“This commit will ...”

Examples:

- Add project README
- Fix typo in setup instructions
- Add data cleaning script
- Update figure caption

9 Part 4: Edit, Diff, Stage, Commit

Open the file again:

```
code README.md
```

Add one more bullet and a takeaway sentence:

```
- comparing versions
```

```
My main takeaway: Git saves meaningful snapshots of a project.
```

Save the file, then check status:

```
git status
```

Use `git diff` to see what changed before staging:

```
git diff
```

Stage the file:

```
git add README.md
```

Use `git diff --staged` to see what is ready to commit:

```
git diff --staged
```

Commit the change:

```
git commit -m "Expand README learning goals"  
git log --oneline
```

10 Part 5: What Is a Commit?

A commit is a saved snapshot of the repository at a point in time.

Each commit has:

- a unique identifier called a hash,
- an author,
- a date,
- a message,
- a pointer to the previous commit,
- the content changes recorded by Git.

View the history:

```
git log  
git log --oneline  
git log --oneline --graph --decorate --all
```

i Snapshot, not save button

Git is not an automatic save system. Editing a file is not enough. You must decide when a change is worth saving as a commit.

11 Part 6: The Staging Area

The staging area lets you choose exactly what goes into the next commit.

This is useful when you changed several files but want to commit them as separate ideas.

Example:

```
git add README.md
git commit -m "Update README"

git add analysis.py
git commit -m "Add analysis script"
```

One commit should usually represent one logical change.

 Avoid giant commits

Update stuff is not a helpful commit message.
A better commit history is a sequence of small, understandable changes.

12 Part 7: Branches

A branch is a movable pointer to a line of work.

Use branches when you want to work on a feature, experiment, or fix without disturbing the stable version.

Common commands:

```
git branch
git switch -c notes
git switch main
git merge notes
```

Create a branch:

```
git branch
git switch -c notes
git branch
```

Add a new file on the branch:

```
code notes.md
```

Type this content and save:

Class Notes

Important Git ideas:

- The working tree contains files I edit.
- The staging area contains changes selected for the next commit.
- A commit is a snapshot with a message.
- A branch is a line of work.

Stage and commit:

```
git add notes.md
git commit -m "Add class notes"
git log --oneline --graph --decorate --all
```

Switch back to main:

```
git switch main
ls
git log --oneline --graph --decorate --all
```

The `notes.md` file may disappear from the folder view because it exists on the `notes` branch but has not yet been merged into `main`.

Merge the branch into `main`:

```
git merge notes
git log --oneline --graph --decorate --all
ls
```

After a successful merge, delete the branch name if it is no longer needed:

```
git branch -d notes
git branch
```

13 Part 8: GitHub Remote Workflow

A remote is another copy of the repository, usually on GitHub.

The usual first-time GitHub workflow is:

1. Create a repository on GitHub.
2. Copy the repository URL.
3. Connect your local repository to the GitHub repository.
4. Push your commits.

Commands:

```
git remote add origin https://github.com/YOUR-USER/YOUR-REPO.git
git remote -v
git push -u origin main
```

After the first push, you can usually use:

```
git push
```

! Do not paste these commands blindly

Replace YOUR-USER and YOUR-REPO with your actual GitHub username and repository name.

The instructor should demonstrate these commands live because they require a real GitHub account and authentication.

14 Part 9: Clone an Existing Repository

To clone means to download a repository and its history.

```
cd ~/Documents
git clone https://github.com/YOUR-USER/YOUR-REPO.git
cd YOUR-REPO
git status
```

Use clone when:

- joining an existing project,
- downloading your own repository onto a new computer,
- starting from a course repository created by an instructor.

💡 Clone location

Run `pwd` before `git clone` so you know where the new repository folder will be created.

15 Part 10: Pull and Push

Once a repository has a remote, two commands become part of daily work.

`git pull` gets changes from GitHub and integrates them into your local branch.

`git push` uploads your local commits to GitHub.

Typical daily rhythm:

```
git status
git pull

# edit files

git status
git add .
git commit -m "Describe the change"
git push
```

 Pull before starting work

In group projects, pull before editing. This reduces the chance that your local work conflicts with changes already pushed by someone else.

16 Part 11: Pull Requests

A pull request is a discussion around proposed changes.

Despite the name, a pull request usually means:

“Please review my branch and merge it into the main branch.”

Typical GitHub collaboration workflow:

```
git switch main
git pull
git switch -c improve-readme

# edit files

git add README.md
git commit -m "Improve README"
git push -u origin improve-readme
```

Then on GitHub:

1. Open a pull request from `improve-readme` into `main`.
2. Ask for review.
3. Discuss comments.
4. Update the branch if needed.
5. Merge the pull request.
6. Delete the branch after merging.

17 Part 12: Merge Conflicts

A conflict happens when Git cannot automatically combine changes.

This often happens when two people edit the same lines of the same file.

Conflict markers look like this:

```
<<<<<< HEAD
Your current version
=====
Incoming version
>>>>>> branch-name
```

To resolve a conflict:

1. Open the file.
2. Decide what the final text should be.
3. Remove the conflict markers.
4. Save the file.
5. Stage the resolved file.
6. Commit the resolution.

Commands:

```
git status

# edit conflicted files

git add conflicted-file.md
git commit -m "Resolve merge conflict"
```

💡 Conflict mindset

A conflict does not mean Git is broken. It means Git needs a human decision.

18 Part 13: .gitignore

Some files should not be committed.

Examples:

- temporary files,
- cache folders,
- large generated outputs,
- secrets and passwords,
- local environment folders,
- operating system files.

Create a .gitignore file:

```
code .gitignore
```

Example .gitignore content:

```
# Temporary files
*.tmp
*.log

# Python cache
__pycache__/
.ipynb_checkpoints/

# Local environment files
.env

# Rendered Quarto artifacts
*_files/
*_cache/
.quarto/
```

Stage and commit .gitignore:

```
git status
git add .gitignore
git commit -m "Add gitignore rules"
git status
```

! Never commit secrets

Do not commit passwords, API keys, private tokens, database credentials, or private personal data.

If a secret has already been committed, deleting it in a later commit is not enough. The secret still exists in the repository history and should be rotated.

19 Part 14: Safe Undo Commands

Git has powerful undo commands. Some are safer for beginners than others.

Goal	Safer command
See what changed	<code>git diff</code>
See staged changes	<code>git diff --staged</code>
Unstage a file but keep the edit	<code>git restore --staged FILE</code>
Discard uncommitted edits in a file	<code>git restore FILE</code>
Create a new commit that reverses an old commit	<code>git revert COMMIT</code>
Move to another branch	<code>git switch BRANCH</code>

Commands to use carefully:

Command	Why careful
<code>git reset --hard</code>	discards local changes
<code>git clean -fd</code>	deletes untracked files and folders
force push	rewrites shared remote history

Practice unstaging:

```
touch scratch.txt
git add scratch.txt
git status
git restore --staged scratch.txt
git status
```

The file is no longer staged, but it still exists in the working tree.

20 Part 15: Reading History

Useful history commands:

```
git log --oneline
git log --oneline --graph --decorate --all
git show HEAD
git show HEAD~1
git diff HEAD~1 HEAD
```

Vocabulary:

Term	Meaning
HEAD	the commit currently checked out
HEAD~1	one commit before HEAD
HEAD~2	two commits before HEAD
hash	a commit identifier
branch	a name pointing to a commit
tag	a permanent label for an important commit

21 Part 16: Working With VS Code

VS Code can help beginners see Git state visually.

Useful VS Code features:

- Source Control panel,
- file change indicators,
- side-by-side diffs,
- staging and unstaging buttons,
- commit box,
- branch selector,
- terminal inside the project folder.

Even when using the interface, learn the command-line meaning:

VS Code action	Git concept
Stage Changes	<code>git add</code>
Commit	<code>git commit</code>
Sync Changes	usually pull then push
Create Branch	<code>git switch -c</code>
Publish Branch	<code>git push -u origin branch-name</code>

💡 Best beginner habit

Use VS Code to inspect changes visually, but keep `git status` as your truth-check command.

22 Part 17: GitHub Issues and Project Work

GitHub is not only a place to store code. It also helps coordinate work.

Issues are useful for:

- bug reports,
- feature requests,
- tasks,
- discussion,
- assignment tracking.

A good issue includes:

- the goal,
- the current problem,
- steps to reproduce if there is a bug,
- expected result,
- relevant screenshots or files,
- owner and deadline if used for a team.

Example issue title:

Add installation instructions for Windows students

Example issue body:

The README explains macOS setup but not Windows setup.

Tasks:

- Add Git installation link.
- Explain Git Bash vs PowerShell.
- Add a screenshot of the VS Code terminal.
- Test the commands on a Windows machine.

23 Part 18: Good Repository Hygiene

A beginner-friendly repository should usually include:

File or folder	Purpose
README.md	project overview and instructions
.gitignore	files Git should ignore
data/	data files, if appropriate
src/ or scripts/	code
notebooks/	exploratory notebooks
docs/	documentation
LICENSE	reuse rules

README checklist:

- What is this project?
- Who is it for?
- How do I install or open it?
- How do I run it?
- What files matter first?
- Where should collaborators ask questions?

24 Part 19: Common Beginner Problems

24.1 “Git says not a git repository”

You are probably in the wrong folder.

Check:

```
pwd
ls -la
git status
```

Look for a `.git` folder. If there is no `.git` folder, either move into the correct repository or initialize a new repository intentionally with `git init`.

24.2 “The command line says no such file or directory”

Possibilities:

- You misspelled a folder or file name.
- You are in the wrong folder.
- The path contains spaces and needs quotes.

Check:

```
pwd
ls
```

If needed, use quotes:

```
cd "folder with spaces"
```

24.3 “code command does not work”

The VS Code command-line shortcut may not be installed.

On macOS, open VS Code and run **Shell Command: Install ‘code’ command in PATH** from the Command Palette.

On Windows, the shortcut is often installed automatically. If it is not available, open the folder from VS Code using **File > Open Folder**.

24.4 “Nothing to commit”

Possibilities:

- You did not save the file.
- You are in the wrong repository.
- You already committed the change.
- The file is ignored by `.gitignore`.

Check:

```
git status
git log --oneline
git check-ignore -v FILENAME
```

24.5 “Rejected push”

Usually the remote has commits that you do not have locally.

Try:

```
git pull
git push
```

If there are conflicts, resolve them, commit the resolution, then push again.

24.6 “Authentication failed”

GitHub does not use normal account passwords for Git operations over HTTPS.

Use one of these:

- browser sign-in through Git Credential Manager,
- GitHub CLI authentication,
- SSH keys,
- a personal access token if required by your environment.

25 Part 20: Responsible AI Use With Git

AI tools can help with Git, but students should still inspect commands before running them.

Good AI prompts:

```
Explain what this Git error means in beginner language.
```

```
I ran git status and got this output. What should I do next?
```

```
Suggest a clear commit message for this diff.
```

Risky prompts:

```
Give me one command to force fix my Git repo.
```

```
Delete all untracked files and reset everything.
```

Before running an AI-suggested command, ask:

- Will this delete local work?
- Will this rewrite shared history?
- Am I on the correct branch?
- Did I run `git status` first?

26 Class Lab

Use this sequence for a 60 to 90 minute beginner workshop. The instructor demonstrates each section in a terminal. Students follow along in their own practice repository.

26.1 Lab A: Local Repository

1. Create a repository.
2. Add a `README.md`.
3. Commit it.
4. Edit it.
5. View the diff.
6. Commit the edit.
7. Read the log.

Core commands:

```
cd ~/Documents
mkdir git-github-lab
cd git-github-lab
git init
git branch -M main
git status
code README.md
git add README.md
git commit -m "Add README"
git diff
git log --oneline
```

26.2 Lab B: Branch and Merge

1. Create a branch called `experiment`.
2. Add a new file.
3. Commit the new file.
4. Switch back to `main`.
5. Merge the branch.

Core commands:

```
git switch -c experiment
code experiment.md
git add experiment.md
git commit -m "Add experiment notes"
git switch main
git merge experiment
git branch -d experiment
```

26.3 Lab C: GitHub

1. Create an empty GitHub repository.
2. Add it as a remote.
3. Push `main`.
4. Confirm the files appear on GitHub.
5. Edit the README on GitHub.
6. Pull the GitHub edit back to the local computer.

Core commands:

```
git remote add origin https://github.com/YOUR-USER/YOUR-REPO.git
git remote -v
git push -u origin main
git pull
```

26.4 Lab D: Pull Request

1. Create a branch.
2. Make a small edit.
3. Push the branch.
4. Open a pull request on GitHub.
5. Merge the pull request.
6. Pull the merged change locally.

Core commands:

```
git switch main
git pull
git switch -c improve-docs
code README.md
git add README.md
git commit -m "Improve documentation"
git push -u origin improve-docs
```

27 Quick Reference

27.1 Terminal Commands

Task	Git Bash	macOS Terminal
Show current folder	<code>pwd</code>	<code>pwd</code>
List files	<code>ls</code>	<code>ls</code>
List hidden files	<code>ls -la</code>	<code>ls -la</code>
Change folder	<code>cd folder</code>	<code>cd folder</code>
Go home	<code>cd ~</code>	<code>cd ~</code>
Go up one level	<code>cd ..</code>	<code>cd ..</code>
Create folder	<code>mkdir folder</code>	<code>mkdir folder</code>
Create file	<code>touch file.txt</code>	<code>touch file.txt</code>

Task	Git Bash	macOS Terminal
Open file in VS Code	<code>code file.txt</code>	<code>code file.txt</code>
Open folder in VS Code	<code>code .</code>	<code>code .</code>
View file	<code>cat file.txt</code>	<code>cat file.txt</code>
Rename or move	<code>mv old.txt new.txt</code>	<code>mv old.txt new.txt</code>
Copy file	<code>cp old.txt copy.txt</code>	<code>cp old.txt copy.txt</code>
Delete file	<code>rm file.txt</code>	<code>rm file.txt</code>
Clear screen	<code>clear</code>	<code>clear</code>
Open file manager	<code>explorer .</code>	<code>open .</code>

27.2 Git Commands

Task	Command
Check state	<code>git status</code>
Start tracking a repository	<code>git init</code>
Rename current branch to main	<code>git branch -M main</code>
Stage a file	<code>git add FILE</code>
Stage all current changes	<code>git add .</code>
Commit staged changes	<code>git commit -m "Message"</code>
View short history	<code>git log --oneline</code>
View graph history	<code>git log --oneline --graph --decorate --all</code>
View changes before staging	<code>git diff</code>
View staged changes	<code>git diff --staged</code>
List branches	<code>git branch</code>
Create and switch branch	<code>git switch -c BRANCH</code>
Switch branch	<code>git switch BRANCH</code>
Merge branch into current branch	<code>git merge BRANCH</code>
Delete merged branch	<code>git branch -d BRANCH</code>
Add GitHub remote	<code>git remote add origin URL</code>
Show remotes	<code>git remote -v</code>
Push first time	<code>git push -u origin main</code>
Push after first time	<code>git push</code>
Pull latest remote changes	<code>git pull</code>
Clone repository	<code>git clone URL</code>
Unstage a file	<code>git restore --staged FILE</code>
Discard local edits in a file	<code>git restore FILE</code>

28 Exit Ticket

Answer these questions at the end of class:

1. What is the difference between Git and GitHub?
2. What does `pwd` tell you?
3. What does `git status` tell you?
4. What is the difference between staging and committing?
5. Why should commit messages be specific?
6. What is a branch useful for?
7. What should you do before starting work in a shared repository?
8. What should you never commit to Git?

29 Suggested Homework

Create a GitHub repository for a small personal learning project.

Requirements:

1. The repository has a `README.md`.
2. The repository has at least three commits.
3. The commit messages are specific.
4. The repository has a `.gitignore`.
5. A branch was created and merged.
6. The final version is pushed to GitHub.

Submit:

- the GitHub repository URL,
- a screenshot of `git log --oneline --graph --decorate --all`,
- a short paragraph explaining one mistake or confusion you encountered and how you fixed it.